WARP ACCELERATORS · DEVELOPER NOTES

ARM extensions: running your own code on the Warp's STM32H7

How to build, seal, load and drive an *extension*: a bare-metal program that the 68k side uploads to the Warp1260X / Warp7060 board and runs on its 400–480 MHz Cortex-M7, next to the firmware, with a byte link and a DMA path back to the Amiga. Written for a developer porting arcade games, and for the Claude sessions helping him: everything you need is meant to be on these pages, with pointers into the reference code.

1 What an extension is, and when to use one
2 Requirements
3 Architecture and memory map
4 The `WarpExtEnv` contract
5 Building the ARM side
6 Containers and sealing
7 The 68k side: cswarp.library
8 Designing the link protocol
9 Performance on the M7
10 Debugging and tools
11 A minimal skeleton, both halves
12 Rules and pitfalls (the checklist)

Status as of 11 October 2026 · firmware 30.52 or newer · cswarp.library 30.41 or newer · `WarpExtEnv` version 4
Sources of truth: `csAmigaWarp/Arm/common/WarpExtEnv.h` (the contract), `csWarpAmigaSoftware/cswarp_library/sdk_includes/include/libraries/cswarp.h` (the 68k API). When this document and those headers disagree, the headers win.
Reference implementations: `WarpQuakeARM` (Quake's renderer on the ARM) and `warpVideoPlay` (MPEG-1/2 decoder on the ARM). Both are private cs-lab repositories: ask Andrzej for access.

1 · What an extension is, and when to use one

A Warp accelerator carries an STM32H753 (Cortex-M7 at 400 MHz, 480 MHz on some boards; double-precision FPU) that runs the board firmware: USB, networking, SD, audio, the update machinery. On the boards with SDRAM (Warp1260X and Warp7060) the firmware sets aside 14 MB of that SDRAM and an 8 MB render area for one *extension*: a program built separately with Arm Compiler 6, uploaded by a 68k program through `cswarp.library`, and started on a thread of its own. It runs until it returns, or the 68k stops it.

The extension and the 68k program talk over:

- **A byte link**: two 64 KB rings, one each way. The 68k calls `extXchg()`, which moves up to 3584 bytes each way per call. The ARM calls `env->send()` / `env->recv()`. What the bytes mean is entirely up to you. Use it for commands, input state, small results and log text.
- **A DMA path** for bulk data: the ARM registers up to 16 buffers, and the 68k pulls them with `extFetch()` (board → Warp RAM or RTG VRAM, rectangle-capable, so straight onto a screen) or pushes into them with `extFeed()` (Warp RAM → board). Both go through the card's QSDMA engine, not through the 68k.
- **A doorbell**: when the 68k has nothing to read, it can sleep on an exec signal that the board raises when the extension sends something or ends.

Why this matters for arcade ports

Many arcade boards of the 1980s and 90s were far stronger than a 68060 at the jobs they were built for: tile and sprite engines with many layers and per-line scroll, rotation/zoom hardware, several CPUs and sound chips running together, DSPs. A pure 68k port has to emulate all of that on the 68k. An extension lets you split the work:

Good candidates for the ARM
Video chip emulation (tilemaps, sprites, priority mixing, line scroll, rotation/zoom), producing a finished frame that the 68k copies to the screen with one `extFetch()`. Sound chip emulation (FM, PCM, ADPCM) producing sample blocks. A second or third emulated CPU (Z80, 6809, a DSP). Decompression of assets. Anything that is a tight integer or float loop over a few hundred KB of state.

Better left on the 68k
Talking to the OS: files, the screen, input, audio output (the ARM has no file system and no AmigaOS access). Code that native 68k already runs fast, such as an original 68000 game program running natively. Anything that needs to exchange many tiny messages per frame: each `extXchg()` is a round trip through the board.

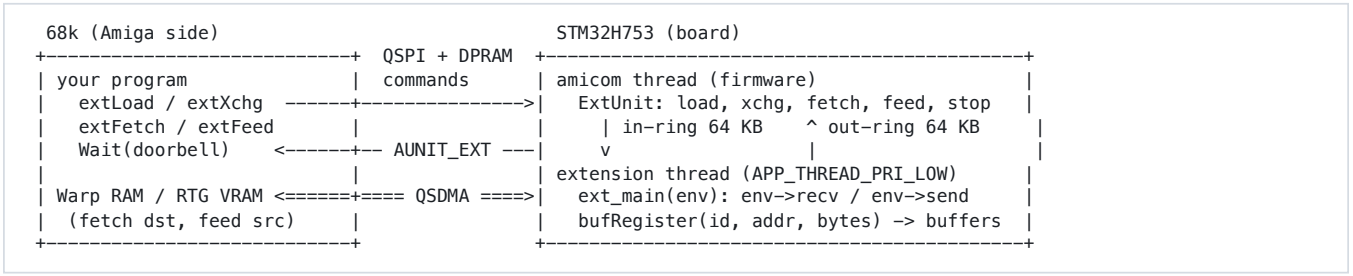
The two programs we have built so far show the shape. In **WarpQuakeARM** (split mode) the 68k runs the game and the ARM runs the renderer. Frames come back by `extFetch()`, and the M7 renders 640×400 at about 28 fps on a 7060. In **warpVideoPlay** the ARM decodes MPEG-1/2. The 68k feeds it the file with `extFeed()`, fetches decoded RGB565 pages straight into an off-screen bitmap, and plays the sound itself.

One extension at a time. The board runs a single extension. A second `extLoad()` while one runs returns `CSWEXT_ERR_BUSY`. Stop yours with `extStop()` on every exit path of the 68k program, Ctrl-C and failures included, or the next program (or the next run of yours) finds the board busy.

2 · Requirements

What	Detail
Board	**Warp1260X or Warp7060.** They are the only boards with the SDRAM the window lives in. On Warp560/1240/1260, `extLoad()` returns `CSWEXT_ERR_NODEV`.
Firmware	Extensions exist from firmware 30.41. **Use 30.52 or newer**: it adds the fast on-chip memory (`WarpExtEnv` v4) and the sealed-container check. Release firmware of 30.52+ **runs sealed containers only** (section 6).
cswarp.library	30.41 or newer (`CSWARP_EXT_MIN_VER/_REV`). The library in the board's ROM shadows any copy in `LIBS:`, so check what is actually loaded with `Version cswarp.library FULL`. Only a boot ROM update refreshes it.
ARM compiler	**Arm Compiler 6** (`armclang`, `armlink`, `fromelf`), 6.24 tested. We use the standalone CMSIS SDK in `~/cmsis-sdk` (its `armcc/bin`), not the copy that VS Code's CMSIS extension keeps in `~/.vcpkg`. AC6 is x86_64-only: on Apple silicon it runs under Rosetta (the Makefiles prefix `arch -x86_64` by themselves). A macOS upgrade can remove Rosetta; the symptom is "Bad CPU type in executable", and `softwareupdate --install-rosetta` fixes it. GCC is not a drop-in: the image format depends on armlink's `--bare_metal_pie` output.
68k compiler	vbcc with NDK 3.2 (as for all our Amiga code), plus the `cswarp.library` SDK headers from `csWarpAmigaSoftware/cswarp_library/sdk_includes/include`.
Python 3	For `mkwxt.py` (container builder), with the `cryptography` package to seal: `pip3 install cryptography`. For J-Link tools: `pylink-square` (we keep it in a venv).
Extension key	`csAmigaWarp/Arm/common/ext_key_warp.h`, git-ignored. Needed to make containers that release firmware accepts. **Get it from Andrzej directly; never commit it, paste it, or print it** (section 6).
Optional	A SEGGER J-Link plus a debug firmware build, for running extensions without the Amiga and for PC sampling (section 10).

3 · Architecture and memory map



Memory the extension gets

Region	Address	Size	Properties and use
Window	`0xC0200000`–`0xC1000000`	14 MB	SDRAM, cacheable (write-back), executable. The first 64 KB is the J-Link mailbox. A container loaded by the 68k sits at `0xC0210000`, so its image starts at `0xC0210100`. After the image come its bss, the stack (default 64 KB, or the `stackKB` you pass to `extLoad`), and then everything else is the **arena** (`env->arena/arenaSize`), about 13 MB minus your image.
Render area	`0xC1000000`	8 MB	SDRAM, cacheable, write-back **without write-allocate**. A store that misses goes straight to SDRAM instead of first reading the line in. Use it for frame buffers, z-buffers and other buffers written with partial-line or strided stores. Reads still allocate. `env->renderArena/renderArenaSize`.
Fast memory (v4)	reported in `env->fastMem[]`	~120 KB total	On-chip RAM the firmware leaves unused, fastest first. On current firmware: ITCM ~34 KB (`EXEC|TCM`: single-cycle, no cache, code may run there), D2 SRAM2 64 KB (`CACHED`), SRAM3 ~25 KB (`CACHED`). **Never hard-code these addresses.** They move when the firmware grows; read them from the env.

Nothing is allocated for you inside these regions: they are handed over whole, and you manage them yourself (a bump allocator is usually enough). Nothing survives the extension: the next load starts from zeroed bss and fresh rings.

Threads, priority and timing

- The extension runs on one ThreadX thread at `APP_THREAD_PRI_LOW`. That is below every firmware worker and below the amicom thread that serves the 68k, so an extension computing flat out still leaves USB, network and the link working. The other side of this: when the firmware is busy (USB storage traffic, networking), your thread gets less CPU. Budget about **20 % firmware overhead** under load.
- There are no interrupts and no other threads for you. Write the extension as one loop. Use `env->waitIn(ms)` to sleep until the 68k sends something, so you do not spin while idle.
- `env->cycles()` is the DWT cycle counter at `env->cpuHz`. Use it for precise timing, and `env->millis()` for coarse time (it keeps counting when a debugger detaches and stops the DWT).
- Byte order: the ARM is **little-endian**; the 68k is big-endian. The rings and the DMA move bytes unchanged. Section 8 covers what to do about that.

The life of an extension

1. 68k: `extLoad(container, size, args, stackKB, req)`. The library copies the container into the window. The firmware checks the header and authenticates and decrypts a sealed container (about 22 ms per 330 KB). It then checks the CRC, applies the relocations in place, zeroes the bss over the relocation table, cleans the D-cache and invalidates the I-cache, and starts the thread at the entry point. The state goes to `EXTST_RUNNING`, or to `EXTST_REJECTED` with an `EXTERR_*` code.
2. ARM: `ext_main(env)` runs. It registers its buffers, then loops: receive, compute, send.
3. 68k: loops on `extXchg()`, `extFetch()` / `extFeed()` and sleeping on the doorbell.
4. The end: either the ARM returns from `ext_main` (state `EXTST_EXITED`, with `exitCode` set to its return value), or the 68k calls `extStop(timeoutMs, req)`. That sets `env->stopRequested()` and wakes `waitIn()`; if the extension has not returned within the timeout, the firmware kills the thread (`EXTST_ENDED`). An Amiga reset also ends it.

4 · The `WarpExtEnv` contract

The entry point is `int ext_main(const WarpExtEnv *env)`. Its return value becomes `exitCode`. The struct is defined in `Arm/common/WarpExtEnv.h` (plain C, `<stdint.h>` only). Copy that header into your project or point your include path at the firmware repo. The struct is **append-only**: fields are never moved or removed, new ones are added at the end, and `version` is bumped. Before you touch a field newer than v1, check both `env->version` and `env->size`.

v	Field	Meaning
1	`uint32_t version, size`	The firmware's `WEXT_ENV_VERSION` and its `sizeof(WarpExtEnv)`.
1	`uint32_t cpuHz`	Core clock (400 MHz, or 480 MHz on boards whose chip allows it): the rate of `cycles()`. Never assume a value.
1	`void log(const char *)`	A NUL-terminated line to the firmware console (RTT) and the loader's log. Slow-ish; not for per-frame use.
1	`uint32_t cycles(void)`	DWT CYCCNT, free-running, wraps (every ~10.7 s at 400 MHz).
1	`void *arena; uint32_t arenaSize`	Your heap: cacheable SDRAM after the stack, about 13 MB.
1	`const void *data; uint32_t dataSize`	A blob loaded next to the extension. **Only the J-Link runner sets it**; with `extLoad` it is NULL. Send data over the link instead.
1	`const char *args`	The command line passed to `extLoad` (at most 1024 bytes, NUL-terminated).
1	`cacheClean(addr, bytes)`, `cacheInvalidate(addr, bytes)`	D-cache maintenance by range (32-byte lines). You do **not** need it around `extFetch`/`extFeed`: the firmware does that for registered buffers.
1	`uint32_t millis(void)`	Milliseconds since boot.
2	`void *renderArena; uint32_t renderArenaSize`	The 8 MB no-write-allocate area (section 3).
3	`uint32_t send(const void *, uint32_t len)`	Puts up to `len` bytes into the out-ring; **never blocks**; returns how many it took (possibly 0 when the 68k is behind). Rings the doorbell if the 68k is waiting.
3	`uint32_t recv(void *, uint32_t max)`	Takes up to `max` bytes from the in-ring; never blocks; returns the count.
3	`int waitIn(uint32_t timeoutMs)`	Sleeps until the in-ring has data (1), the time is up (0), or a stop was requested (1).
3	`int bufRegister(uint32_t id, void *addr, uint32_t bytes)`	Makes memory reachable to the 68k's `extFetch`/`extFeed` under `id` 0..15. `addr` must be 4-byte aligned and lie in the window or the render area (not in fast memory). Returns 0 on refusal.
3	`int stopRequested(void)`	Non-zero after the 68k's `extStop`. Poll it in every long loop and return soon after it turns non-zero.
4	`uint32_t fastMemCount; WarpExtMem fastMem[4]`	Free on-chip RAM, fastest first: `{base (32-byte aligned), size, flags}`, flags `WEXT_MEM_EXEC 1`, `WEXT_MEM_CACHED 2`, `WEXT_MEM_TCM 4`.

The link functions (v3) are NULL when the extension was started by the J-Link runner rather than by a 68k program. If you also want to run under the J-Link runner (section 10), test them before use.

```
/* the version checks every extension should make */
#include <stddef.h>
#define ENV_HAS(e, field) ((e)->size >= offsetof(WarpExtEnv, field) + sizeof((e)->field))

int linked = env->version >= 3 && ENV_HAS(env, stopRequested) && env->send != NULL;
int fast   = env->version >= 4 && ENV_HAS(env, fastMem) ? env->fastMemCount : 0;
```

5 · Building the ARM side

An extension is a **bare-metal position-independent executable** (PIE): linked at address 0, with a list of words that the loader adds the load address to. It has no C library start-up, no file system and no `main`. Its globals are reached PC-relative through a GOT, which is why it can run wherever the firmware puts it.

Compile and link flags

```
# compile (every .c)
armclang --target=arm-arm-none-eabi -mcpu=cortex-m7 -mfpu=fpv5-d16 -mfloat-abi=hard -mthumb \
         -fbare-metal-pie -ffunction-sections -std=gnu11 \
         -O3 -fno-unroll-loops [-ffp-mode=fast] -include ext_stdio.h -I<fw>/Arm/common -c x.c -o x.o

# link
armlink --cpu=Cortex-M7.fp.dp --bare_metal_pie --no_startup --library_type=microlib \
        --entry=ext_main --diag_suppress=6771,6084 --remove \
        --map --list=build/arm/myext.map --info=sizes,totals  *.o -o build/arm/myext.axf

# container
python3 mkwxt.py build/arm/myext.axf build/arm/myext.wxt --name myext --key <fw>/Arm/common/ext_key_warp.h
```

- `-fbare-metal-pie` at compile time *and* `--bare_metal_pie` at link time are both required. **LTO cannot be used**: it drops `-fbare-metal-pie`.
- `-fno-unroll-loops`: the image runs from SDRAM through a 16 KB I-cache, and an I-cache miss costs about 146 cycles. Unrolled code that does not fit the cache is slower. Keep this flag in any `OPT` override; a test script that dropped it once faked a 1 fps "regression".
- `-ffp-mode=fast` where the code tolerates it (it did for Quake). The FPU is double precision, but single precision is still faster and halves the memory.
- Use `--library_type=microlib`. Microlib supplies `memcpy`, `memset`, the maths functions and `sprintf`-style formatting. Do not call anything that needs an OS: `malloc`, stdio files, `exit`, `time`, `clock`.
- `--diag_suppress=6771`: "the entry point is not in the first execution region"; harmless here. `6084`: armlink warns about sections it cannot place by attribute; harmless for the PIE image.

> **The `__arm_relocate_pie_` stub.** With `--bare_metal_pie`, armlink insists on the C library's relocation routine, and microlib does not have it. Define an empty one; the firmware applies the relocations itself (`mkwxt.py` extracts them for it), so it is never called:
>
> ```
> void __arm_relocate_pie_(void) { }
> ```

Porting existing C: the `ext_stdio.h` / `ext_libc.c` pattern

An emulator core expects `malloc`, `fopen`, `printf` and `exit`. Rather than edit the core, both reference projects force a header in front of every source file (`-include ext_stdio.h`). The header `#define`s those names to `ext_*` functions in `ext_libc.c`:

C call	Replacement in `ext_libc.c`
`malloc/calloc/free`	A heap carved from `env->arena`. A bump allocator plus a free list is enough for an emulator that allocates at start-up.
`fopen/fread/fseek…`	An in-memory file set: ROM images that the 68k sends over the link (or `extFeeds`) at start-up. Writes (save states, high scores) collect in memory, and the 68k is asked to store them.
`printf/fprintf(stderr)`	Formats into a line buffer and calls `env->log` (and/or sends a LOG record to the 68k).
`exit`	`longjmp` back into `ext_main`, which returns the code.

The same sources also build for the host (macOS clang, `make host`), with a small `host_main.c` that fakes a `WarpExtEnv` (heap arena, file-backed rings). **Do this from day one.** Most bugs are found on the host, with a debugger and sanitizers, before anything goes to the board. WarpQuakeARM's host build even runs the whole split, both halves, in one process.

> Start a new project by copying `warpVideoPlay/arm/`'s `Makefile`, `ext_stdio.h`, `ext_libc.c/.h`, `mkwxt.py`, `fastcheck.py` and `host_main.c`, then replace `ext_main.c` and the sources. The Makefile already handles Rosetta, the AC6 location (`AC6=`, `CMSIS_SDK=`), the key path (`FW_COMMON=`, `EXT_KEY=`) and a separate optimisation level for third-party code.

What the image must not do

- Touch hardware registers, enable or disable interrupts, change the MPU or the caches globally, or call ThreadX. You are a guest thread in the firmware. Use only what `env` offers.
- Write outside the window, the render area and the reported fast memory. There is no MPU fence around you; a stray pointer corrupts the firmware, and the usual symptom is the whole board freezing.

- Busy-wait on the 68k. Use `waitIn()`.
- Use unaligned 16/32-bit accesses to *non-cacheable* memory. Cacheable SDRAM (the window, the render area) tolerates them. Data in the ring and buffers you register is in cacheable memory, so normal C is fine there.

6 · Containers and sealing

`mkwxt.py` turns the linked `.axf` into a `.wxt` container:

```
offset 0     WarpExtHeader (256 bytes)  magic "WEXT", containerVer, imageSize, bssSize, entryOff (Thumb bit),
                                        relocCount, envVersion, imageCrc, name[32], keyId, sealedSize, nonce[12], tag[16]
offset 256   image                       the load region as linked at address 0
             relocs                      relocCount × uint32 image offsets; the loader adds the image address to each word
             (bss is not stored: the loader zeroes bssSize bytes after the image, over the reloc table)
```

`mkwxt.py` also checks every relocation target lies inside the image and holds an address inside it, so a format surprise fails on your machine and not on the board.

Sealed containers (containerVer 2)

With `--key ext_key_warp.h`, the image and relocations (zero-padded to 16 bytes) are encrypted with **AES-256-GCM** under the board's extension key and a random 96-bit nonce. The header, with the tag field zeroed, is authenticated as additional data. The GCM tag is the signature: only someone holding the key can make a container the board will run, and only the board can read its contents. The board decrypts with the STM32's hardware CRYP unit before anything runs.

Firmware build	Plain container (v1)	Sealed container (v2)
Release (what users and you normally have)	refused: `EXTERR_SIGNATURE`	runs if the tag verifies with its key
Debug (built for J-Link work)	runs	runs if the tag verifies

A refused container comes back to the 68k as `extLoad() == CSWEXT_ERR_REJECTED` (0x41) with `req.exitCode == EXTERR_SIGNATURE` (6). It also happens when the container was sealed with a different key generation (`keyId`), or when the file was modified after sealing. Without the key file, `mkwxt.py` writes a plain container and prints a warning, so a missing key is easy to overlook: read the build output.

> **Key handling (important for Claude sessions too).** `ext_key_warp.h` is a secret shared by every Warp board. It is git-ignored in `csAmigaWarp`. Keep it there, or at a path outside any repository, and point `EXT_KEY=` at it. Never commit it, never copy it into your project repo, never paste or print its contents (no `cat`, no quoting it in a chat or a log), and never ship it with a game. The key is separate from the firmware-update key, and it can be rotated by a normal firmware update (`WEXT_KEY_ID` is bumped). If that happens, re-seal your containers with the new file.

Embedding the container in your program

Ship one executable, not a `.wxt` file that can get out of step with it. Both reference programs link the container in as data with a two-line vasm file and pass that to `extLoad`:

```
; wxt_blob.s -- the board half, linked into the 68k program
        section wxt,data
        xdef    _myWxt
        xdef    _myWxtEnd
_myWxt:
        incbin  "../arm/build/arm/myext.wxt"
_myWxtEnd:
```

```
extern const UBYTE myWxt[], myWxtEnd[];
rc = extLoad(myWxt, myWxtEnd - myWxt, args, 64, &req);
```

Make the 68k program's Makefile depend on the `.wxt` through a recipe that always re-runs the ARM build (`$(WXT): FORCE` with `$(MAKE) -C arm`), so that a rebuilt extension is never left out of the executable. Keep a command-line override that loads a container from a file (WarpQuakeARM: `-armclient <file>`; warpVideoPlay: `WXT=`). It is handy for testing a new ARM build without relinking.

7 · The 68k side: cswarp.library

Open `cswarp.library` version 30, then check the revision. Include `<libraries/cswarp.h>` and `<proto/cswarp.h>` from the SDK. All extension calls take an optional `CSWarpExtReq *`, which every call fills with the extension's state. **Zero it and set** `version = CSWARP_EXTREQ_VERSION` **before the first call.** Every call returns a `CSWEXT_*` result.

Call (LVO)	What it does
`extLoad(container, size, args, stackKB, req)` (–558)	Uploads the container and starts it with the command line `args` (max 1024 bytes; `""` for none). `stackKB` 0 means the 64 KB default. The container may sit anywhere in 68k memory, at any alignment.
`extXchg(req)` (–564)	One exchange: sends up to `req.sendLen` bytes of `req.send` (max 3584; `req.sent` = what the in-ring took) and receives up to `req.recvMax` into `req.recv` (max 3584; `req.received`). Send 0 with `recvMax` 0 to just ask for the state.
`extFetch(buf, offset, dst, rowBytes, rows, pitch, req)` (–570)	Copies `rows × rowBytes` from registered buffer `buf` at `offset` into Warp RAM or VRAM at `dst`, with rows `pitch` bytes apart. This lets the ARM's packed frame go straight into a screen bitmap.
`extFeed(buf, offset, src, len, req)` (–576)	Copies `len` bytes from Warp RAM at `src` into buffer `buf` at `offset`.
`extStop(timeoutMs, req)` (–582)	Asks the extension to end. After `timeoutMs` (at most 5000) it is ended anyway. Call it on every exit path.

Rules for `extFetch` / `extFeed`

- `dst/src`, `rowBytes`, `pitch` and `len` must be **multiples of 16**. The total must be **at least 32 bytes**, and `offset` a multiple of 4. `pitch ≥ rowBytes`. The buffer must have been registered by the ARM; the transfer must fit inside it.
- The 68k address must be in the **Warp's own RAM or its RTG VRAM**: QSDMA cannot reach chip RAM or Zorro memory, and the library refuses with `CSWEXT_ERR_BADPARAM`. For a P96 bitmap, lock it to get the address and `BytesPerRow`, and check that both are 16-aligned. Make the bitmap a little wider if needed. Do not make OS calls while holding a P96 lock.
- The bytes are copied unchanged. The ARM must store pixels in the byte order the Amiga screen expects (for example big-endian RGB565 for `RGBFB_R5G6B5`: use `__builtin_bswap16` or `__rev16` while rendering, which is free on the M7).
- The calls are synchronous: they return when the data has arrived, with caches handled on both sides. A 640×400×16-bit frame (500 KB) takes a few milliseconds. Measure it with `req.boardUs`.
- The ARM must not write a page while the 68k is fetching it. Use two or more pages and a "done with page N" message (section 8).

Results and states

Result	Meaning
`CSWEXT_OK`	done
`CSWEXT_ERR_NODEV`	no board, no SDRAM, or firmware too old
`CSWEXT_ERR_BUSY`	`extLoad`: one is already running
`CSWEXT_ERR_NOMEM`	the container does not fit
`CSWEXT_ERR_NOTREADY`	nothing running
`CSWEXT_ERR_REJECTED` 0x41	refused; `req.exitCode` = `EXTERR_*`
`CSWEXT_ERR_BADPARAM`	alignment, range, buffer id
`CSWEXT_ERR_IO`, `_DMA` 0x40	link or QSDMA failure

`req.state`	`EXTERR_*` **(on REJECTED)**
0 `NONE`	1 `HEADER`: bad magic, size, relocation
1 `LOADING`	2 `CRC`: image corrupt
2 `RUNNING`	3 `LAYOUT`: image + bss + stack don't fit
3 `EXITED` (returned)	4 `ENV`
4 `ENDED` (stopped)	5 `THREAD`: thread not created
5 `REJECTED`	6 `SIGNATURE`: not sealed / wrong key

The other fields filled on every call are `inFree` (room in the in-ring), `outPending` (bytes the ARM sent that you have not received yet), `bufMask` (registered buffers), `windowBytes/renderBytes` and `boardUs`. `bufMask` is how the 68k learns that the ARM has registered its buffers. Wait for the right bits, or for a hello message, before the first `extFetch`.

The main loop and the doorbell

Polling `extXchg()` in a tight loop works, but it takes the 68k away from everything else: in warpVideoPlay, an extra exchange per frame once cost 30 % of the frame rate. Do one exchange per pass, and sleep when nothing moved. The doorbell rule (from `cswarp.h`): **after a reply with** `received == 0` **and** `outPending == 0`**, the board rings** `CSWARP_UNIT_EXT` **once the extension sends or ends. Wait for the signal only then.**

```
BYTE doorSig = AllocSignal(-1);
unitSigRegister(FindTask(NULL), CSWARP_UNIT_EXT, doorSig);   /* after extLoad succeeded */
for (;;) {
    req.send = txBuf;  req.sendLen = txLen;  req.recv = rxBuf;  req.recvMax = sizeof(rxBuf);
```

```
    if (extXchg(&req) != CSWEXT_OK) break;
    consume(txBuf, req.sent);                    /* keep what the ring did not take */
    handleRecords(rxBuf, req.received);
    if (req.state != EXTST_RUNNING) break;       /* exited / ended */
    if (req.received || req.outPending || req.sent) continue;  /* something moved: go again */
    /* idle: the doorbell is armed now. Sleep on it plus your other signals, with a timer as a safety net */
    got = Wait((1UL << doorSig) | timerMask | SIGBREAKF_CTRL_C | windowMask);
    if (got & SIGBREAKF_CTRL_C) break;
}
extStop(1000, &req);
unitSigUnregister(FindTask(NULL), CSWARP_UNIT_EXT);
FreeSignal(doorSig);
```

Stale signals. `WaitIO()` leaves the timer port's signal set, so the next `Wait()` returns at once. Clear it with `SetSignal(0, 1UL << port->mp_SigBit)` before you re-arm the timer (see warpVideoPlay's `sleepFor()`). `Wait()` also clears a Ctrl-C it returns: if another part of your program checks for a break, put it back with `SetSignal(SIGBREAKF_CTRL_C, SIGBREAKF_CTRL_C)`.

8 · Designing the link protocol

The rings carry bytes; the framing is yours. The format we use in both projects has worked well. Copy it and change the record types:

- A record is `u8 type, u8 flags, u16 length` of the payload, then the payload, padded to a multiple of 4.
- **Every multi-byte field is little-endian**, the ARM's native order, so the ARM side reads structs directly. The 68k side swaps with two small helpers (`get32/put32`). Keep one shared header (`common/xxx_link.h`) that both compilers include. Use plain C only, with no structs on the wire: write fields byte by byte, so vbcc and armclang can never disagree on layout or packing.
- Type numbers below 16 for 68k→ARM and 16 and up for ARM→68k make traces easy to read.
- Start with a **HELLO** carrying a protocol version, and have the other side refuse a mismatch. A 68k program and an embedded container built from different commits are the commonest bug.
- Include a **LOG** record (ARM→68k text). It is the most convenient debug channel: no J-Link needed, and the text shows up in the 68k program's console.
- Both `send()` and the 68k's ring take partial writes. Never drop the remainder: keep it queued and send it on the next pass. On the ARM, when `send()` returns less than asked, the 68k is behind. Call `waitIn(1..5)` and retry, checking `stopRequested()`.
- For bulk data, use a record that *describes* it ("frame in page 1", "the 68k fed N bytes at offset X") and move the data with `extFetch/extFeed`. The 3584-byte `extXchg` limit is for messages, not frames.

A protocol for an arcade port (sketch)

Dir	Record	Payload and meaning
68k→ARM	`HELLO`	u32 protocol version, u32 options (screen depth, sound rate)
68k→ARM	`ROM`	u32 region id, u32 size: announces a ROM region the 68k then `extFeed`s into buffer `BUF_ROM`
68k→ARM	`INPUT`	u32 joystick/button bitmap, u32 frame number: sent once per frame
68k→ARM	`DONE`	u32 page: "fetched; you may draw into it again"
68k→ARM	`QUIT`	(none): the ARM answers with `END` and returns
ARM→68k	`READY`	u32 width, height, stride, pages: buffers are registered
ARM→68k	`FRAME`	u32 page, u32 frame number: page holds a finished frame
ARM→68k	`AUDIO`	u32 buffer id, u32 offset, u32 samples: a block of samples ready to fetch
ARM→68k	`LOG`	text
ARM→68k	`END`	u32 reason, statistics

For timing, let the 68k own the clock: it knows the display's refresh and the audio position. It sends `INPUT` when it wants the next frame, and it can tell the ARM to skip rendering when it falls behind (warpVideoPlay's `PACE`). The ARM then never needs to know about Amiga time.

9 · Performance on the M7

The M7 is fast at arithmetic and slow at memory that is not cached. Every number here was measured on a Warp7060 or 1260X:

Fact	Consequence
SDRAM is 16 bits wide; a cache miss costs about **366 ns ≈ 146 cycles**	The extension is memory-bound long before it is CPU-bound. Count cache misses, not instructions.
D-cache 16 KB, I-cache 16 KB, 32-byte lines	Keep hot tables small; prefer `uint8_t/uint16_t` tables; keep code compact (`-fno-unroll-loops`; compile cold third-party code with `-O2`).
ITCM: single-cycle, code and data	Put the hottest inner loop there (a tile/sprite blitter, a CPU core's dispatch). WarpQuakeARM went from 25.7 to 28.0 fps with ITCM and SRAM2.
SRAM2/SRAM3: on-chip, cached, much faster on a miss than SDRAM	Use them for data walked once per frame that misses the cache (span lists, a line buffer, sprite lists, sound-chip state).
Render area: no write-allocate	Frame buffers there cost no line reads for stores. Write whole words or doublewords (pack two RGB565 pixels into one `uint32_t` store): byte stores to SDRAM are slow.
Firmware overhead about 20 % under load	Do not size a design that needs 100 % of the core.

Running code from ITCM

Code can be copied into the fast memory only if it references **nothing outside itself**, because the PIE image reaches globals and other functions PC-relative, and a copy at another address would get those offsets wrong. The pattern from `d_arm.c`:

```
#define FAST __attribute__((section("hotcore"), noinline))

typedef struct { const uint8_t *tiles; uint16_t *dst; const uint16_t *pal; int n; } BlitCtx;
FAST static void BlitCore(BlitCtx *c) { /* uses only c->..., locals, and no calls */ }

static void (*blit)(BlitCtx *) = BlitCore;          /* SDRAM copy until moved */

extern char hotBase[]  __asm__("hotcore$$Base");
extern char hotLimit[] __asm__("hotcore$$Limit");
#define MOVED(f, dst) ((void *)((uintptr_t)(dst) + (((uintptr_t)(f) & ~1u) - (uintptr_t)hotBase) + 1))

void HotInit(const WarpExtEnv *env) {                 /* pick a region with WEXT_MEM_EXEC|WEXT_MEM_TCM */
    void *itcm = ...;  uint32_t n = hotLimit - hotBase;
    memcpy(itcm, hotBase, n);
    __asm__ volatile("dsb\n\tisb" ::: "memory");
    blit = MOVED(BlitCore, itcm);                      /* the wrapper fills a BlitCtx and calls blit() */
}
```

Only copy code into regions flagged `WEXT_MEM_EXEC`; today that is the ITCM only. Check the result with `fastcheck.py build/arm/my-ext.axf hotcore fromelf`, which disassembles the section and fails on any branch, call, literal load or `ADD pc` that leaves it. Put it in the Makefile before `mkwxt.py`, as both reference projects do. Avoid `switch` statements that compile to tables outside the section, and calls to `memcpy` or helpers (division, 64-bit shifts) inside a core. `fastcheck` catches these.

Data needs none of this: allocate it from a fast region with a bump allocator, using `fastMem[i].base` and `size`. Fall back to the arena when it is full or `fastMemCount` is 0.

10 · Debugging and tools

Without a debugger (what you will use most)

- **The host build** first: the same sources against a fake env on macOS. Use lldb, AddressSanitizer and frame-by-frame CRC checks.
- **LOG records** to the 68k console, and `env->log()` to the firmware console.
- **WarpDebug** (in `csWarpAmigaSoftware`) checks the mechanism from a Shell, independently of your program: `WarpDebug extLoad=<file.wxt> extArgs="..."`, `WarpDebug extState` (state, exit code and any text it sent), `WarpDebug extStop`. Use it to clear a board left busy by a crashed program.
- **Frame CRCs**: have the ARM send a CRC of each frame and compare it with the host build. This is the fastest way to find layout and byte-order bugs. If the emulated game uses randomness tied to timing (sound), make the run deterministic first. In Quake, sound changed `rand()` and cost hours.

With a J-Link (debug firmware)

A debug firmware build has a J-Link mailbox at the start of the window (`Arm/csArmedWarp/Src/extproto.h`), plus the tools in `WarpQuakeARM/arm/` and `warpVideoPlay/arm/` (Python + `pylink`):

Tool	Use
`wxrun.py`	Runs an extension with no Amiga program at all: `wxrun.py build/arm/myext.wxt --data roms.bin --args "..."`. It loads the container and a blob (`env->data`), starts it and streams its log. The link functions are NULL in this mode. Good for benchmarking a core in isolation.
`wxsample.py` + `wxsym.py`	A statistical profiler without halting the CPU: samples the PC through DWT_PCSR thousands of times a second, then symbolises against your `.axf` (`--base 0xC0210100` for a 68k-loaded extension) and the firmware's `.axf`. It tells you where the time actually goes.
`wxpeek.py`	The J-Link mailbox's state, plus a crude halt-and-read PC profile.
Ozone	The firmware's Ozone project (`csWarpedARM_ozone.jdebug`) for the firmware side. Your code has no symbols there; map addresses by hand with your `.map` file and the image base.

> **Three J-Link traps.** (1) **J-Link reads bypass the D-cache**: data the M7 just wrote may still be in the cache, so a dump shows stale SDRAM. Have the extension call `env->cacheClean()` on the range first. (2) The firmware's RTT buffer is 1 KB and **drops everything once full** if nothing reads it: drain it (set RdOff to WrOff) before a test. (3) **Detaching the debugger stops the cycle counter** for up to ~100 ms. Do not sample a run that times itself with `cycles()`, and use `millis()` for anything spanning a detach. A HardFault with a debugger attached spins in the handler, and the whole board, the 68k link included, stops answering.

Symptoms and what they usually mean

Symptom	Usual cause
`extLoad` → 0x41, exitCode 6	The container is not sealed (key file missing at build time: read the `mkwxt` output), sealed with an older key, or modified after sealing.
`extLoad` → 0x41, exitCode 3	Image + bss + stack do not fit: a huge static array. Move it to the arena or the render area.
`extLoad` → `BUSY`	The previous run was not stopped. Run `WarpDebug extStop`, and add `extStop` to your exit paths.
Whole board / Amiga freezes	The extension wrote outside its memory (a stray pointer, an overrun), or a HardFault with a debugger attached.
Stuck at state `EXITED` at once, exitCode −1/…	Your own early return: the env version check failed, or allocation failed. Send a LOG before every early return.
Garbage colours / swapped pixels	Byte order: the ARM wrote little-endian RGB565 to a big-endian screen.
`extFetch` → `BADPARAM`	Something not 16-aligned (the P96 `BytesPerRow`!), a buffer not yet registered, or the destination not in Warp RAM/VRAM.
A new ARM build "changes nothing"	The executable still embeds the old container: check the Makefile dependency (`FORCE`).
Slower than expected	Sample it. It is usually cache misses on a table walked per pixel, or byte stores to SDRAM. Check the board's load too: another extension may be running.

11 · A minimal skeleton, both halves

An "emulated video chip" that renders a frame on request into one of two pages and tells the 68k which one to fetch. This is an untested sketch to start from; the reference projects are the tested versions of each part.

ARM: `ext_main.c`

```
#include <stddef.h>
#include <stdint.h>
#include <string.h>
#include "WarpExtEnv.h"
#include "../common/arc_link.h"          /* records: ARC_HELLO, ARC_INPUT, ARC_DONE, ARC_QUIT, ARC_READY, ARC_FRAME,
ARC_LOG */

#define W 320
#define H 224
#define STRIDE (W * 2)                    /* RGB565, a multiple of 16 */
#define PAGES 2

void __arm_relocate_pie_(void) { }       /* armlink wants it; the firmware relocates */

static const WarpExtEnv *env;
static uint8_t rx[4096];
static uint32_t rxLen;

static void put32(uint8_t *p, uint32_t v) { memcpy(p, &v, 4); }       /* LE is native */
static uint32_t get32(const uint8_t *p) { uint32_t v; memcpy(&v, p, 4); return v; }

static int sendRecord(uint8_t type, const uint32_t *words, uint32_t n)
{
  uint8_t rec[4 + 4 * 8];
  uint32_t len = 4 + 4 * n, done = 0, i;
  rec[0] = type; rec[1] = 0; rec[2] = (uint8_t)(4 * n); rec[3] = (uint8_t)((4 * n) >> 8);
  for (i = 0; i < n; i++) put32(rec + 4 + 4 * i, words[i]);
  while (done < len) {                              /* the 68k may be behind: never drop the rest */
    done += env->send(rec + done, len - done);
    if (done < len) { if (env->stopRequested()) return 0; env->waitIn(2); }
  }
  return 1;
}

static void renderFrame(uint16_t *page, uint32_t input) { /* the emulated video chip: write big-endian pixels, two per
uint32 store */ }

int ext_main(const WarpExtEnv *e)
{
  uint16_t *page[PAGES];
  uint32_t busy[PAGES] = { 0 }, cur = 0, frame = 0, i;
  env = e;
  if (e->version < 3 || e->size < offsetof(WarpExtEnv, stopRequested) + sizeof(void *) || !e->send)
    return -1;                                      /* too old, or started without a 68k link */
  for (i = 0; i < PAGES; i++) {
    page[i] = (uint16_t *)((uint8_t *)e->renderArena + i * STRIDE * H);
    if (!e->bufRegister(i, page[i], STRIDE * H)) return -2;
  }
  { uint32_t r[4] = { W, H, STRIDE, PAGES }; sendRecord(ARC_READY, r, 4); }

  while (!e->stopRequested()) {
    uint32_t off = 0;
    if (!e->waitIn(100)) continue;                  /* idle: nothing from the 68k */
    rxLen += e->recv(rx + rxLen, sizeof(rx) - rxLen);
    while (rxLen - off >= 4) {
      uint32_t plen = rx[off + 2] | rx[off + 3] << 8, rlen = (4 + plen + 3) & ~3u;
      const uint8_t *p = rx + off + 4;
      if (rxLen - off < rlen) break;                /* partial record: wait for the rest */
      switch (rx[off]) {
      case ARC_INPUT:
        if (!busy[cur]) {                           /* else the 68k is behind: drop this frame */
          renderFrame(page[cur], get32(p));
          busy[cur] = 1;
          { uint32_t r[2] = { cur, frame++ }; sendRecord(ARC_FRAME, r, 2); }
          cur = (cur + 1) % PAGES;
        }
        break;
      case ARC_DONE: if (get32(p) < PAGES) busy[get32(p)] = 0; break;
      case ARC_QUIT: return 0;
      }
      off += rlen;
    }
```

```
    memmove(rx, rx + off, rxLen - off);
    rxLen -= off;
  }
  return 0;
}
```

68k: the loader (vbcc)

```
#include <proto/exec.h>
#include <proto/cswarp.h>
#include <libraries/cswarp.h>
#include <string.h>

struct CSWarpBase *csWarpBase;
extern const UBYTE myWxt[], myWxtEnd[];            /* wxt_blob.s */
static CSWarpExtReq req;

int main(void)
{
  LONG rc;
  csWarpBase = (struct CSWarpBase *)OpenLibrary("cswarp.library", CSWARP_EXT_MIN_VER);
  if (!csWarpBase) return 20;
  if (((ULONG)csWarpBase->libNode.lib_Version << 16 | csWarpBase->libNode.lib_Revision)
      < ((ULONG)CSWARP_EXT_MIN_VER << 16 | CSWARP_EXT_MIN_REV)) { /* too old */ }

  memset(&req, 0, sizeof(req));
  req.version = CSWARP_EXTREQ_VERSION;
  rc = extLoad(myWxt, myWxtEnd - myWxt, "", 0, &req);
  if (rc == CSWEXT_ERR_REJECTED && req.exitCode == EXTERR_SIGNATURE) { /* not sealed for this firmware */ }
  if (rc != CSWEXT_OK) goto out;

  /* open the screen; allocate the frame destination in Warp RAM / VRAM, 16-aligned;
     register the doorbell; loop as in section 7: extXchg -> on FRAME(page):
       extFetch(page, 0, dst, 320 * 2, 224, bytesPerRow, &req); then send DONE(page);
     once a frame (vblank / timer): send INPUT(joystick bits). */

  extStop(1000, &req);                              /* always, on every exit path */
out:
  CloseLibrary((struct Library *)csWarpBase);
  return 0;
}
```

12 · Rules and pitfalls (the checklist)

Hand this section to a Claude session working on an extension. Each line was learned on real hardware.

Build

1. Compile with `-fbare-metal-pie -mcpu=cortex-m7 -mfpu=fpv5-d16 -mfloat-abi=hard -mthumb -ffunction-sections`; link with `--bare_metal_pie --no_startup --library_type=microlib --entry=ext_main`. No LTO.
2. Define an empty `__arm_relocate_pie_`.
3. Keep `-fno-unroll-loops` in every optimisation setting, including A/B test scripts.
4. Seal with `mkwxt.py --key`, and read its output: without the key it silently makes a plain container that release firmware refuses.
5. **Never print, commit, copy into a project, or paste** `ext_key_warp.h` (nor `aes_key_warp.h`). Refer to it by path only.
6. Embed the container with `incbin`, and make the 68k build depend on it through a `FORCE`d recipe.
7. Run the host build before every board test.

ARM side

8. Check `env->version` *and* `env->size` before using a v2+ field. The link functions are NULL under the J-Link runner.
9. Large buffers go in the arena or the render area, not in static arrays (bss counts against the window layout: `EXTERR_LAYOUT`).
10. Never hard-code fast-memory addresses; read `fastMem[]`. Copy code only into `EXEC` regions, and only code that passes `fastcheck.py`.
11. Never busy-wait: `waitIn()`. Poll `stopRequested()` in every loop that can run longer than ~100 ms.
12. `send()` may take less than you gave it; keep the rest. Never block forever on a full ring: check `stopRequested()`.
13. Registered buffers: in the window or the render area, 4-aligned. Don't write a page the 68k may be fetching; use a DONE handshake.
14. Pixels and every value the 68k reads from a fetched buffer must be stored big-endian. Values in link records are little-endian, by convention.
15. Store frames in words or doublewords, not bytes. Put stride-written buffers in the render area.
16. No hardware registers, no ThreadX calls, no interrupt or cache-control instructions beyond `dsb`/`isb` after copying code.

68k side

17. Check the library version and revision against `CSWARP_EXT_MIN_VER/_REV`; zero `CSWarpExtReq` and set its version.
18. `extStop()` and `unitSigUnregister()` on every exit path, Ctrl-C and errors included.
19. `extFetch`/`extFeed`: 16-aligned addresses, row bytes, pitch and length; at least 32 bytes; Warp RAM or VRAM only.
20. One `extXchg` per loop pass. Sleep on the doorbell only after a reply with nothing received and `outPending` 0. Clear stale timer signals before `Wait()`.
21. No OS calls (not even `Printf`) while holding a P96 bitmap lock.
22. Exchange a protocol version in the first message and refuse a mismatch.

Testing

23. Check what is loaded: `Version cswarp.library FULL` (the ROM copy shadows `LIBS:`), and the firmware version.
24. A board left with an extension running: `WarpDebug extState` / `extStop`.
25. Make benchmark runs deterministic (input recording, no timing-dependent randomness), and compare frame CRCs with the host build.
26. When J-Link memory dumps contradict the code, suspect the D-cache before the code.

Where to look

File	What
`csAmigaWarp/Arm/common/WarpExtEnv.h`	The env and container contract (authoritative).
`csAmigaWarp/Arm/csArmedWarp/Src/extunit.cpp/.h`	The firmware loader: unsealing, relocation, the rings, fetch/feed, stop.
`csAmigaWarp/Arm/csArmedWarp/Src/extproto.h`	The J-Link mailbox protocol (debug firmware).
`cswarp_library/sdk_includes/include/libraries/cswarp.h`	The 68k API, result codes, request struct, the doorbell rule.
`WarpDebug/src/dbg_ext.c`	The smallest working 68k client.
`warpVideoPlay/arm/`, `/amiga/`, `/common/wvp_link.h`	The cleanest complete example: Makefile, ext_libc, fast-memory allocator, player loop, doorbell, fetch to a P96 bitmap, feed from a file.
`WarpQuakeARM/arm/` (`d_arm.c`, `ext_qg.c`, tools)	ITCM cores and `fastcheck.py`, the profiler tools, `wxrun.py`, the split game/renderer.